

Multi-Theme Firmware Engine Architecture

Multi-Theme Firmware Engine

Multi-Theme Firmware Engine Architecture

Status: Compile-passing multi-theme firmware candidate. Hardware validation pending.
This is the integrated 20-theme engine foundation. Current themes differentiate via palette, labels, and LED ring color. Further visual refinement (per-theme widget layouts matching the rich concept prototypes) may be needed after hardware testing and contact-sheet review.
Goal: one compile-passing firmware with all 20 selectable themes, not final visual perfection.

1. Overview

The VelaDial door-side controller runs a single, unified firmware containing all 20 UI themes. This is a fundamental shift from "one YAML per concept" to a "shared engine with dynamic rendering" approach.

2. Shared State Architecture

To avoid duplicating control logic 20 times, the firmware will rely on a core set of shared globals:

```
globals:
- id: active_theme
  type: int
  initial_value: "0"
  restore_value: yes # Crucial for persistence

- id: current_page
  type: int
  initial_value: "0" # 0=Power, 1=Brightness, 2=Presets

- id: display_awake
  type: bool
  initial_value: "true"

- id: brightness_pct
  type: int
  initial_value: "50"

- id: lights_on
  type: bool
  initial_value: "false"
```

3. Shared Action Handlers

All physical inputs (knob rotation, knob press, screen touch, swipes) will route through shared scripts. These scripts enforce the "wake-only-first" rule and then execute the appropriate action based on the `current_page` and `active_theme`.

- `handle_knob_cw` / `handle_knob_ccw`
- `handle_knob_press`
- `handle_power_toggle`
- `send_brightness`
- `apply_preset`

4. Dynamic Rendering Engine

Instead of creating 60 separate LVGL pages (20 themes × 3 pages), we will use a tiered rendering approach to conserve RAM and Flash memory.

The Core Pages

We will maintain the 3 core pages (Power, Brightness, Presets) plus 1 new page (Theme Selector).

Reusable LVGL Primitives

We will define a set of generic LVGL widgets on each page: - `power_bg_circle` - `power_main_label` - `power_sub_label` - `brightness_main_arc` - `brightness_sub_arc` - `brightness_pct_label`

The `render_active_theme` Script

When the theme changes, or when the device wakes, a master script (`render_active_theme`) will execute. This script uses a large `if/then` or `switch` structure based on `id(active_theme)` to modify the properties of the reusable primitives.

For example, if Theme 01 (Minimal) is active, `brightness_main_arc` is set to blue and 8px wide. If Theme 19 (Vinyl) is active, `brightness_main_arc` is set to amber, 14px wide, and additional concentric circles are made visible.

5. Memory Management Strategy

- **Hide, don't delete:** Widgets specific to certain themes (e.g., the 5 glow rings for Eclipse Corona) will be created once but set to `hidden: true` when not in use.
- **Shared Fonts:** We will strictly limit fonts to `roboto16`, `roboto24`, `roboto32`, and `roboto48` to save flash space.
- **No Images:** All visuals will be generated using LVGL primitives (arcs, lines, circles, rectangles) rather than heavy image assets.